

Documentacion de cambios y funcionalidad del backend

Informe tecnico del estado implementado despues de la auditoria. Describe la arquitectura, seguridad, reglas de negocio, API REST, configuracion, pruebas y trabajo pendiente.

Tecnologias	Java 21, Spring Boot 4.1, Spring Security, JPA, Flyway, MySQL/H2, React 19, Vite 8
Enfoque	Backend y cambios de seguridad/consistencia realizados
Fecha	29 de julio de 2026
Estado verificado	Frontend compila; backend compila; prueba de contexto aprobada en Docker/JDK 21

Alcance: este documento refleja el codigo disponible en el repositorio. No contiene secretos reales ni presupone funcionalidades que todavia no estan implementadas.

1. Resumen ejecutivo

La aplicacion gestiona tickets de soporte para la UIFCE. El backend expone una API REST para autenticacion, usuarios, tickets, categorias, subcategorias, edificios y oficinas. La persistencia usa JPA; MySQL es la base de produccion y H2 se usa en pruebas/desarrollo.

Cambios mas importantes

- Autenticacion coherente: login por contrasena y Google usan la sesion de Spring Security.
- CSRF activo: el frontend obtiene una cookie XSRF y envia la cabecera correspondiente.
- Autorizacion por roles: operaciones administrativas se protegen en backend con @PreAuthorize.
- Identidad confiable: crear o modificar tickets usa el usuario autenticado, no un ID suministrado por el cliente.
- Privacidad de tickets: Usuario ve los propios; Tecnico y Administrador pueden consultar todos.
- Configuracion segura: OAuth se configura mediante variables de entorno.
- Contrato corregido: la ruta para crear subcategorias ahora coincide entre frontend y backend.
- Limpieza: se retiraron dependencias, scripts, respaldos y codigo muerto demostrablemente innecesarios.

Resultado aproximado: 1.500 lineas eliminadas y cuatro dependencias directas menos.

Contenido

Seccion	Contenido
2	Arquitectura y recorrido de una solicitud
3	Autenticacion, sesiones, CSRF y roles
4	Funcionalidad del backend por modulo
5	Catalogo de endpoints
6	Modelo de datos y reglas de tickets
7	Configuracion, Docker y ejecucion
8	Pruebas y verificaciones
9	Inventario de cambios
10	Limitaciones y siguientes pasos

2. Arquitectura del sistema

El backend sigue una estructura por capas sencilla. Los controladores reciben HTTP y validan DTO; los servicios aplican reglas de negocio; los repositorios Spring Data acceden a entidades JPA; SecurityConfig controla sesion, CSRF y acceso.

Cliente React -> Nginx / CORS -> Spring Security -> Controller -> Service -> Repository -> MySQL/H2

Responsabilidades

Capa	Responsabilidad	Ejemplos
Configuracion	Seguridad, CORS, Flyway y beans comunes.	SecurityConfig, CorsConfig, FlywayConfig
Controladores	Mapeo REST, DTO, codigos HTTP y autorizacion declarativa.	AuthController, TicketController
Servicios	Reglas, transacciones y conversion a respuestas.	AuthService, TicketService
Dominio	Entidades, enums, calculos y relaciones.	Ticket, User, Category, Office
Persistencia	Consultas y paginacion mediante Spring Data JPA.	TicketRepository, UserRepository
Infraestructura	Errores, validacion compartida y contexto de auditoria.	ErrorHandler, EntityValidator, AuditContextAspect

Recorrido: crear un ticket

- El cliente obtiene/incluye el token CSRF y envia POST /api/v1/tickets.
- Spring Security exige una sesion autenticada.
- TicketController obtiene el correo desde la autenticacion; ignora identidades del cliente.
- TicketService busca usuario, subcategoria y oficina; crea Ticket y calcula puntaje, prioridad y tiempo.
- TicketRepository persiste la entidad y el controlador responde 201 Created.

Recorrido: listar tickets

- Administrador o Tecnico recibe una pagina global.
- Usuario recibe solo tickets cuyo user.email coincide con la sesion.
- La ultima asignacion se consulta para incluir el tecnico actual en el DTO de respuesta.

3. Seguridad implementada

3.1 Login por contraseña

AuthService normaliza el correo, exige el dominio institucional, comprueba que el usuario exista y este activo, y valida BCrypt. AuthController crea un UsernamePasswordAuthenticationToken con la autoridad del rol y guarda el SecurityContext en la sesion HTTP.

```
POST /api/auth/login
{ "email": "persona@unal.edu.co", "password": "..."}
-> 200 { "user": { "id": 1, "nombre": "...", "email": "...", "rol": "Usuario" } }
```

3.2 Login con Google

CustomOAuth2UserService delega en Google, exige un correo verificado terminado en @unal.edu.co, busca o crea el usuario y asigna ROLE_Usuario por defecto. El parametro hd mejora la seleccion de cuenta, pero la validacion real ocurre despues de recibir los atributos verificados.

3.3 CSRF y cookies

- GET /api/auth/csrf materializa el token CSRF.
- Spring envia XSRF-TOKEN; el frontend responde con X-XSRF-TOKEN.
- Todas las llamadas usan credentials: incluye para conservar la sesion.
- Las mutaciones sin token valido son rechazadas por Spring Security.

3.4 Matriz de autorizacion

Capacidad	Usuario	Tecnico	Administrador
Crear ticket propio	Si	Si	Si
Consultar tickets	Solo propios	Todos	Todos
Cambiar estado	No	Si	Si
Asignar tecnico	No	No	Si
Listar usuarios	No	Si	Si
Crear/desactivar usuarios	No	No	Si
Consultar catalogos	Si	Si	Si
Modificar categorias/ubicaciones	No	No	Si

Importante: los controles del frontend mejoran la experiencia, pero la autoridad final es el backend.

4. Funcionalidad del backend por modulo

4.1 Usuarios

- Crear usuarios con nombre, correo y rol.
- Listar usuarios paginados para Tecnico o Administrador.
- Consultar el usuario de la sesion mediante /usuarios/me.
- Consultar por ID y desactivar usuarios, reservado al Administrador.

4.2 Tickets

- Crear tickets asociados al usuario autenticado, subcategoria y oficina.
- Calcular puntaje a partir de subcategoria, oficina y cantidad de equipos.
- Derivar prioridad y tiempo esperado de respuesta.
- Listar tickets con paginacion y tecnico asignado.
- Asignar tecnico y registrar asignaciones.
- Cambiar estado respetando transiciones y comentario obligatorio al cerrar.

4.3 Categorías y subcategorías

- Crear, consultar, actualizar y desactivar categorías.
- Crear subcategorías dentro de una categoría y administrar su puntaje.
- Listar subcategorías por categoría.

4.4 Edificios y oficinas

- Administrar edificios con numero y nombre.
- Administrar oficinas hijas y su puntaje.
- Desactivar y reactivar oficinas.

4.5 Auditoria

Existe una entidad Audit y un aspecto que establece variables de contexto SQL para los triggers. Sin embargo, AuditController permanece vacio y el frontend no consulta auditorias reales. El registro en base puede existir, pero la API de consulta todavia no esta implementada.

4.6 Manejo de errores

EntityValidator centraliza la busqueda de entidades y evita repetir comprobaciones. ErrorHandler transforma excepciones de validacion, entidades inexistentes y argumentos invalidos en respuestas HTTP consistentes.

5. Catalogo de endpoints

5.1 Autenticacion

Metodo	Ruta	Acceso	Funcion
GET	/api/auth/csrf	Publico	Entrega/materializa token CSRF.
POST	/api/auth/login	Publico + CSRF	Valida credenciales y crea sesion.
GET	/oauth2/authorization/google	Publico	Inicia OAuth2 con Google.
GET	/login/oauth2/code/google	Google	Callback procesado por Spring Security.

5.2 Usuarios

Metodo	Ruta	Rol	Funcion
POST	/api/v1/usuarios	Administrador	Crea usuario.
GET	/api/v1/usuarios	Tecnico/Admin	Lista usuarios paginados.
GET	/api/v1/usuarios/usuarios_registrados	Tecnico/Admin	Lista usuarios finales paginados.
GET	/api/v1/usuarios/me	Autenticado	Devuelve identidad de sesion.
GET	/api/v1/usuarios/{id}	Administrador	Consulta usuario por ID.
DELETE	/api/v1/usuarios/{id}	Administrador	Desactiva usuario (borrado logico).

5.3 Tickets

Metodo	Ruta	Rol	Funcion
POST	/api/v1/tickets	Autenticado	Crea ticket para la identidad de sesion.
GET	/api/v1/tickets	Autenticado	Pagina global para Tecnico/Admin o propia para Usuario.
PATCH	/api/v1/tickets/cambiar-estado	Tecnico/Admin	Cambia estado aplicando reglas.
PATCH	/api/v1/tickets/asignar-tecnico	Administrador	Asigna un tecnico.

Paginacion: los endpoints paginados aceptan parametros estandar de Spring, por ejemplo
?page=0&size=20&sort=fechaCreacion,desc, si el nombre de propiedad coincide con la entidad.

5.4 Categorías y subcategorías

Metodo	Ruta	Rol	Funcion
POST	/api/v1/categoria	Administrador	Crea categoria.
GET	/api/v1/categoria	Autenticado	Lista categorias.
GET	/api/v1/categoria/{id}	Autenticado	Detalle de categoria.
PUT	/api/v1/categoria	Administrador	Actualiza categoria.
DELETE	/api/v1/categoria/{id}	Administrador	Desactiva categoria.
POST	/api/v1/categoria/{id}/subcategoria	Administrador	Crea subcategoria.
GET	/api/v1/categoria/{id}/subcategorias	Autenticado	Lista hijas de categoria.
GET	/api/v1/categoria/subcategoria/{id}	Autenticado	Detalle de subcategoria.
PUT	/api/v1/categoria/subcategoria	Administrador	Actualiza subcategoria.
DELETE	/api/v1/categoria/subcategoria/{id}	Administrador	Desactiva subcategoria.

5.5 Edificios y oficinas

Metodo	Ruta	Rol	Funcion
POST	/api/v1/edificios	Administrador	Crea edificio.
GET	/api/v1/edificios	Autenticado	Lista edificios.
GET	/api/v1/edificios/{id}	Autenticado	Detalle de edificio.
PUT	/api/v1/edificios	Administrador	Actualiza edificio.
DELETE	/api/v1/edificios/{id}	Administrador	Desactiva edificio.
POST	/api/v1/edificios/{id}/oficinas	Administrador	Crea oficina.
GET	/api/v1/edificios/{id}/oficinas	Autenticado	Lista oficinas hijas.
GET	/api/v1/edificios/oficinas/{id}	Autenticado	Detalle de oficina.
PUT	/api/v1/edificios/oficinas	Administrador	Actualiza oficina.
DELETE	/api/v1/edificios/oficinas/{id}	Administrador	Desactiva oficina.
PATCH	/api/v1/edificios/oficinas/{id}	Administrador	Reactiva oficina.

6. Modelo de datos y reglas de negocio

Entidades principales

Entidad	Relacion	Proposito
User	1:N Ticket; 1:N Assignment	Identidad, rol, estado y datos OAuth/contrasena.
Ticket	N:1 User, Subcategory, Office	Solicitud, estado, prioridad, fechas, puntaje y comentario.
Category	1:N Subcategory	Familia de servicios.
Subcategory	N:1 Category	Tipo especifico y puntaje.
Building	1:N Office	Sede/edificio.
Office	N:1 Building	Ubicacion y puntaje.
Assignments	N:1 User y Ticket	Historial de asignaciones tecnicas.
Audit	Referencias opcionales	Registro de acciones y detalles JSON.

Estados y transiciones

- Abierto -> En_proceso: flujo normal al tomar/asignar ticket.
- En_proceso -> Cerrado: exige comentario final no vacio.
- Abierto -> Cerrado: prohibido para Tecnico.
- En_proceso -> Abierto: prohibido para Tecnico.
- Reabrir un ticket Cerrado queda reservado al Administrador.

Puntaje y prioridad actuales

Puntaje total	Prioridad	Tiempo de respuesta
1-10	Alta	24 horas
11-20	Alta	48 horas
21-30	Alta	72 horas
31-40	Media	5 dias
41-50	Media	10 dias
51-60	Media	15 dias
61-70	Baja	Durante el semestre academico
> 70	Baja	No establecido

El total suma puntaje de subcategoria, oficina y cantidad de equipos. La cantidad aporta 1 (0-3), 10 (4-14), 20 (15-30) o 35 (>30). El DTO exige una cantidad positiva, por lo que cero no entra desde la API.

7. Configuración, Docker y ejecución

Variables requeridas

Variable	Uso	Observación
GOOGLE_CLIENT_ID	Cliente OAuth2	Obligatoria si se habilita Google.
GOOGLE_CLIENT_SECRET	Secreto OAuth2	No debe versionarse ni imprimirse.
SPRING_DATASOURCE_URL	Conexión MySQL	Docker usa mysql:3306/uifce_support.
SPRING_DATASOURCE_USERNAME	Usuario BD	Evitar root en producción.
SPRING_DATASOURCE_PASSWORD	Clave BD	Usar gestor de secretos.
SPRING_PROFILES_ACTIVE	Perfil Spring	Docker usa production por defecto.
FRONTEND_PORT	Puerto publicado	8080 por defecto.

Arranque con Docker Compose

```
docker compose up --build
# Frontend/Nginx: http://localhost:8080
# MySQL: localhost:3306 (configurable)
# Backend: interno en backend:8080, publicado mediante Nginx
```

Desarrollo local

```
# Backend (requiere JDK 21)
.\mvnw.cmd spring-boot:run -Dspring-boot.run.profiles=dev

# Frontend
cd frontend
npm ci
npm run dev
```

Migraciones

Flyway contiene V1-V7 para crear tablas, triggers de auditoría, ajustar asignaciones, agregar contraseñas y OAuth, y actualizar el hash del administrador. En el perfil test Flyway está deshabilitado y Hibernate crea el esquema H2.

Acción externa pendiente: el secreto OAuth que estuvo expuesto debe revocarse y regenerarse en Google Cloud. Cambiar el archivo local no invalida la credencial ni la elimina del historial Git.

8. Pruebas y verificaciones

Comprobacion	Resultado	Detalle
Build frontend	Aprobado	TypeScript y Vite generan dist de produccion.
Build backend	Aprobado	Compilacion Java 21 dentro de Docker.
Prueba Spring	Aprobado	Contexto con perfil test y H2: 1/1.
git diff --check	Aprobado	Sin espacios o parches mal formados.
ESLint	Pendiente	Errores heredados: hooks, any y reglas de refresh.
npm audit	Advertencia	Aviso alto de React Router para RSC; esta SPA no usa RSC.

9. Inventario de cambios realizados

- Secreto OAuth reemplazado por variables de entorno en configuraciones versionadas y locales.
- application.yaml seguro y versionable; application-local.yaml queda reservado para overrides ignorados.
- LoginResponseDTO devuelve un UserResponseDTO real.
- AuthService elimina token simulado y autentica con BCrypt.
- AuthController crea y persiste el contexto de seguridad de sesion.
- SecurityConfig activa CSRF y elimina el acceso publico al listado de usuarios.
- Controladores protegen mutaciones administrativas con roles.
- TicketController deriva el correo de OAuth2User o de la autenticacion local.
- TicketRepository agrega consulta paginada por correo de propietario.
- TicketService elimina doble guardado y evita confiar en IDs de actor enviados por el cliente.
- Frontend envia cookies/CSRF, usa URL OAuth configurable y normaliza roles.
- Rutas React se agrupan por rol.
- El lockfile se redujo al retirar Express, CORS, dotenv, jsonwebtoken y un visualizador duplicado.
- Se eliminaron logger sin uso, controlador hello, lockfiles duplicados y respaldo SQL obsoleto.
- Se agrego application-test.yaml para ejecutar pruebas sin MySQL ni secretos reales.

10. Limitaciones y siguientes pasos

Prioridad recomendada, de mayor a menor impacto:

Prioridad	Pendiente	Criterio de cierre
Critica	Rotar el secreto OAuth filtrado.	Credencial antigua revocada; nueva cargada como secreto.
Alta	Implementar API de auditoria.	GET paginado, filtros, DTO y autorizacion Admin.
Alta	Pruebas de seguridad y negocio.	Casos 401/403, propiedad de tickets y transiciones.
Alta	Corregir lint del frontend.	npm run lint sin errores.
Media	Completar categorias/ubicaciones en UI.	Guardar/eliminar llama API, no console.log.
Media	Persistir telefono de contacto o retirarlo.	Contrato frontend/backend consistente.
Media	Agregar logout de servidor.	POST protegido por CSRF invalida sesion y cookie.
Media	Desactivar open-in-view.	Servicios cargan lo requerido dentro de transaccion.
Baja	Healthcheck dedicado.	Endpoint minimo o Actuator, sin consultar usuarios.
Baja	CI automatizada.	Build, test, lint y auditorias en cada cambio.

Ejemplos de pruebas minimas necesarias

- Usuario no puede crear/desactivar usuarios ni modificar catalogos.
- Tecnico no puede asignar tecnicos ni reabrir como Administrador.
- Usuario A nunca recibe tickets de Usuario B.
- Crear ticket ignora cualquier identidad suministrada por el cliente.
- Cerrar sin comentario devuelve error; cerrar con comentario registra fecha de resolucion.
- Login invalido no revela si el correo existe.

Criterio de produccion: no desplegar hasta rotar OAuth, cubrir autorizacion con pruebas y confirmar el flujo completo de login/logout sobre HTTPS.

Fin del documento